MAKEMORE · PART 4

Backprop Ninja
the math, explained line by line

Every gradient in Andrej Karpathy's `build_makemore_backprop_ninja` — derived from scratch, with the exact code solution for each line.

A companion to the notebook. Notation: for any tensor T, we write **dT** $= \partial L / \partial T$, the gradient of the scalar loss L with respect to T. It always has the **same shape as** T. Batch size $n = 32$, vocab $= 27$, hidden $= 64$, embedding $= 10$, block $= 3$.

Prepared by **Abdulsamad Teniola Muyideen**.

0. Foundations

Backprop is nothing but the **chain rule** applied mechanically, from the loss backwards to every input. Karpathy "chunks" the forward pass into tiny, one-operation steps precisely so that each backward step is a single, local derivative. Master these six rules and every line below becomes mechanical.

RULE 1 — SAME SHAPE

dT has exactly the same shape as T. If your gradient's shape doesn't match the variable, something is wrong. This is your built-in unit test.

RULE 2 — CHAIN RULE = LOCAL × UPSTREAM

For $b = f(a)$, $\mathbf{da} = \dfrac{\partial b}{\partial a} \cdot \mathbf{db}$. You already received **db** (the upstream gradient) from the step after you; you only supply the **local** derivative $\partial b / \partial a$.

RULE 3 — BROADCASTING FORWARD ⇒ SUMMATION BACKWARD

If a small tensor was **broadcast** (stretched) to be used many times in the forward pass, its gradient is the **sum** of all the places it was used. You sum over exactly the axes that were broadcast, keeping the original shape.

RULE 4 — THE MATRIX-MULTIPLY PATTERN

For $Y = XW + b$ with $X : (n, d_{\text{in}})$, $W : (d_{\text{in}}, d_{\text{out}})$:

$$\mathbf{dX} = \mathbf{dY}\, W^{\top}, \qquad \mathbf{dW} = X^{\top}\mathbf{dY}, \qquad \mathbf{db} = \textstyle\sum_{\text{rows}} \mathbf{dY}.$$

The transposes are forced on you by Rule 1: they are the only arrangement that makes the shapes line up.

RULE 5 — INDEXING / GATHER ⇒ SCATTER-ADD

If the forward pass **selected** rows (e.g. `C[Xb]`), the backward pass **adds** each output's gradient back into the row it came from. Duplicated indices accumulate.

RULE 6 — A VARIABLE USED TWICE ⇒ ADD THE TWO GRADIENTS

If a tensor feeds into two later expressions, its total gradient is the **sum** of the contributions from both branches. In code this is the `+=` you'll see on `dcounts`, `dlogits`, `dbndiff`, and `dhprebn`.

The forward pass we are differentiating

This is the "chunkated" forward pass from the notebook — broken into atomic steps so each can be backpropagated one at a time. Loss flows down; gradients flow back up.

```
# embedding + flatten
emb     = C[Xb]                          # (32, 3, 10)
embcat  = emb.view(emb.shape[0], -1)     # (32, 30)
# Linear layer 1
hprebn  = embcat @ W1 + b1               # (32, 64)
# BatchNorm
bnmeani   = 1/n * hprebn.sum(0, keepdim=True)        # (1, 64)
bndiff    = hprebn - bnmeani                         # (32, 64)
bndiff2   = bndiff**2                                # (32, 64)
bnvar     = 1/(n-1) * bndiff2.sum(0, keepdim=True)   # (1, 64)  Bessel
bnvar_inv = (bnvar + 1e-5)**-0.5                     # (1, 64)
bnraw     = bndiff * bnvar_inv                       # (32, 64)
hpreact   = bngain * bnraw + bnbias                  # (32, 64)
# non-linearity + Linear layer 2
h       = torch.tanh(hpreact)          # (32, 64)
logits  = h @ W2 + b2                  # (32, 27)
# cross-entropy, expanded for stability
logit_maxes    = logits.max(1, keepdim=True).values  # (32, 1)
norm_logits    = logits - logit_maxes                # (32, 27)
counts         = norm_logits.exp()                   # (32, 27)
counts_sum     = counts.sum(1, keepdims=True)        # (32, 1)
counts_sum_inv = counts_sum**-1                      # (32, 1)
probs          = counts * counts_sum_inv             # (32, 27)
logprobs       = probs.log()                         # (32, 27)
loss           = -logprobs[range(n), Yb].mean()      # scalar
```

TENSOR	SHAPE	TENSOR	SHAPE
Xb	(32, 3)	bnvar_inv	(1, 64)
C	(27, 10)	bngain, bnbias	(1, 64)
W1	(30, 64)	h, hpreact	(32, 64)
b1	(64,)	W2	(64, 27)
hprebn, bndiff	(32, 64)	b2	(27,)
bnmeani, bnvar	(1, 64)	logits, probs	(32, 27)

1. Exercise 1 — manual backprop, one variable at a time

We start at the loss and walk *up* the forward pass, producing one gradient per intermediate exactly in reverse order. Each card shows the forward line, the derivation, and the code solution. Watch for Rule 6 (`+=`) whenever a variable was reused.

1 **dlogprobs** (32, 27)

```
loss = -logprobs[range(n), Yb].mean()
```

Only the **correct-class** entry of each row contributes to the loss. Writing it out,

$$L = -\frac{1}{n}\sum_{i=1}^{n} \text{logprobs}[i,\, y_i].$$

So the derivative is $-\frac{1}{n}$ at the picked entries and exactly 0 everywhere else:

$$\frac{\partial L}{\partial \text{logprobs}[i,j]} = \begin{cases} -\frac{1}{n} & j = y_i \\ 0 & \text{otherwise.} \end{cases}$$

We build a zero tensor and write $-1/n$ only into the chosen positions.

SOLUTION

```
dlogprobs = torch.zeros_like(logprobs)
dlogprobs[range(n), Yb] = -1.0/n
```

2 dprobs (32, 27)

```
logprobs = probs.log()
```

Elementwise log. The local derivative of $\log x$ is $1/x$, so by Rule 2:

$$\frac{\partial\,\text{logprobs}}{\partial\,\text{probs}} = \frac{1}{\text{probs}} \quad \Rightarrow \quad \mathbf{dprobs} = \frac{1}{\text{probs}} \odot \mathbf{dlogprobs}.$$

Rows whose target had tiny probability get a **large** gradient — that is the loss pushing hardest where the model was most wrong.

SOLUTION

```
dprobs = (1.0 / probs) * dlogprobs
```

3 dcounts_sum_inv · dcounts (part 1) (32,1) · (32,27)

```
probs = counts * counts_sum_inv
```

Here `counts` is $(32, 27)$ but `counts_sum_inv` is $(32, 1)$, **broadcast** across all 27 columns: $\text{probs}[i,j] = \text{counts}[i,j] \cdot \text{counts\_sum\_inv}[i]$.

For counts (elementwise factor): $\mathbf{dcounts} = \text{counts\_sum\_inv} \odot \mathbf{dprobs}$. This is only the *first* of two contributions — counts is reused in Step 5, so we'll `+=` later (Rule 6).

For counts_sum_inv: it was broadcast, so by Rule 3 we sum its contributions across the 27 columns:

$$\mathbf{d\,counts\_sum\_inv}[i] = \sum_j \text{counts}[i,j] \cdot \mathbf{dprobs}[i,j].$$

SOLUTION

```
dcounts_sum_inv = (counts * dprobs).sum(1, keepdim=True)
dcounts = counts_sum_inv * dprobs   # part 1 of 2
```

4 dcounts_sum (32, 1)

```
counts_sum_inv = counts_sum**-1
```

Power rule on x^{-1}: local derivative $-x^{-2}$. Therefore

$$\mathbf{d\,counts\_sum} = -\,\mathrm{counts\_sum}^{-2} \odot \mathbf{d\,counts\_sum\_inv}.$$

SOLUTION

```
dcounts_sum = (-counts_sum**-2) * dcounts_sum_inv
```

WHY COUNTS_SUM-1 AND NOT 1.0/COUNTS_SUM?**

They are mathematically identical, but PyTorch routes them through different kernels with slightly different floating-point rounding. Using `**-1` is what makes the manual gradient **bit-exact** (maxdiff `0.0`) rather than merely approximate.

5 dcounts (part 2) (32, 27)

```
counts_sum = counts.sum(1, keepdims=True)
```

A sum has local derivative 1 for every term. So each `counts_sum[i]` sends its gradient back, unchanged, to all 27 entries of row i (a broadcast of $(32, 1)$ up to $(32, 27)$):

$$\mathbf{dcounts}[i, j] \mathrel{+}= 1 \cdot \mathbf{d\,counts\_sum}[i].$$

This is the **second** path into `counts`, so we accumulate with `+=` (Rule 6) onto Step 3's result.

SOLUTION

```
dcounts += torch.ones_like(counts) * dcounts_sum
```

6 dnorm_logits (32, 27)

```
counts = norm_logits.exp()
```

The derivative of e^x is e^x — which we already computed as `counts`. So:

$$\mathbf{d\,norm\_logits} = \text{counts} \odot \mathbf{dcounts}.$$

SOLUTION

```
dnorm_logits = counts * dcounts
```

7 dlogits (part 1) · dlogit_maxes (32,27) · (32,1)

```
norm_logits = logits - logit_maxes
```

A subtraction with `logit_maxes` $(32, 1)$ broadcast across columns.

For logits (coefficient $+1$): $\mathbf{dlogits} = \mathbf{d\,norm\_logits}$ — first of two paths (logits is reused in Step 8).

For logit_maxes (coefficient -1, broadcast ⇒ sum across columns):

$$\mathbf{d\,logit\_maxes}[i] = -\sum_j \mathbf{d\,norm\_logits}[i, j].$$

SOLUTION

```
dlogits = dnorm_logits.clone()   # part 1 of 2
dlogit_maxes = (-dnorm_logits).sum(1, keepdim=True)
```

8 dlogits (part 2) (32, 27)

```
logit_maxes = logits.max(1, keepdim=True).values
```

The `max` picks exactly one element per row (its `argmax`); the gradient flows only to that single winning element and is 0 for all the rest. We route it back with a one-hot mask of the argmax indices, and **add** it to Step 7 (Rule 6):

$$\mathbf{dlogits}[i,j] \mathrel{+}= \mathbf{1}\left[j = \arg\max_k \text{logits}[i,k]\right] \cdot \mathbf{d\,logit\_maxes}[i].$$

SOLUTION

```
dlogits += F.one_hot(logits.max(1).indices,
                     num_classes=logits.shape[1]) * dlogit_maxes
```

SANITY CHECK

Subtracting the max is a numerical-stability trick that does not change the loss, so this whole branch's contribution should be negligibly small — but for a *bit-exact* result it must still be included.

9 dh · dW2 · db2 (32,64) · (64,27) · (27,)

```
logits = h @ W2 + b2
```

The classic linear layer — apply Rule 4 directly with $X = h$, $W = W_2$:

$$\mathbf{dh} = \mathbf{dlogits}\, W_2^\top, \quad \mathbf{dW_2} = h^\top \mathbf{dlogits}, \quad \mathbf{db_2} = \textstyle\sum_{\text{rows}} \mathbf{dlogits}.$$

Check the shapes: $(32,27)(27,64) = (32,64)$ for `dh`; $(64,32)(32,27) = (64,27)$ for `dW2`. They could only be arranged this one way.

SOLUTION

```
dh  = dlogits @ W2.T
dW2 = h.T @ dlogits
db2 = dlogits.sum(0)
```

10 dhpreact (32, 64)

```
h = torch.tanh(hpreact)
```

The derivative of tanh is $1 - \tanh^2$, and $\tanh(\text{hpreact})$ is just `h`. So we never need `hpreact` itself:

$$\mathbf{dhpreact} = (1 - h^2) \odot \mathbf{dh}.$$

Saturated neurons ($|h| \to 1$) have $1 - h^2 \to 0$ and pass almost no gradient — this is the vanishing-gradient effect batch-norm is designed to keep in check.

SOLUTION

```
dhpreact = (1.0 - h**2) * dh
```

11 dbngain · dbnraw · dbnbias (1,64) · (32,64) · (1,64)

```
hpreact = bngain * bnraw + bnbias
```

An affine transform with `bngain` and `bnbias` of shape $(1, 64)$ broadcast over the 32 rows.

- **dbnraw** (elementwise factor `bngain`): $\mathbf{dbnraw} = \text{bngain} \odot \mathbf{dhpreact}$.
- **dbngain** (broadcast ⇒ sum over the batch): $\mathbf{dbngain} = \sum_{\text{rows}} \text{bnraw} \odot \mathbf{dhpreact}$.
- **dbnbias** (additive, broadcast ⇒ sum over the batch): $\mathbf{dbnbias} = \sum_{\text{rows}} \mathbf{dhpreact}$.

SOLUTION

```
dbngain = (bnraw * dhpreact).sum(0, keepdim=True)
dbnraw  = bngain * dhpreact
dbnbias = dhpreact.sum(0, keepdim=True)
```

12 dbndiff (part 1) · dbnvar_inv

(32,64) · (1,64)

```
bnraw = bndiff * bnvar_inv
```

A product where `bnvar_inv` $(1, 64)$ is broadcast over the batch.

- **dbndiff** (factor `bnvar_inv`): $\mathbf{dbndiff} = \text{bnvar\_inv} \odot \mathbf{dbnraw}$ — first of two paths (bndiff is reused in Step 15).
- **dbnvar_inv** (broadcast ⇒ sum over batch): $\mathbf{dbnvar\_inv} = \sum_{\text{rows}} \text{bndiff} \odot \mathbf{dbnraw}$.

SOLUTION

```
dbndiff    = bnvar_inv * dbnraw   # part 1 of 2
dbnvar_inv = (bndiff * dbnraw).sum(0, keepdim=True)
```

13 dbnvar

(1, 64)

```
bnvar_inv = (bnvar + 1e-5)**-0.5
```

Power rule on $(x+\varepsilon)^{-1/2}$: local derivative $-\frac{1}{2}(x+\varepsilon)^{-3/2}$. Thus

$$\mathbf{dbnvar} = -\tfrac{1}{2}\,(\text{bnvar} + 10^{-5})^{-3/2} \odot \mathbf{dbnvar\_inv}.$$

SOLUTION

```
dbnvar = (-0.5*(bnvar + 1e-5)**-1.5) * dbnvar_inv
```

14 dbndiff2 (32, 64)

```
bnvar = 1/(n-1) * bndiff2.sum(0, keepdim=True)
```

A scaled column-sum. The local derivative of each summand is the constant $\frac{1}{n-1}$, broadcast back over the 32 rows:

$$\mathbf{dbndiff2}[i,j] = \frac{1}{n-1} \cdot \mathbf{dbnvar}[j].$$

(The $n-1$ is Bessel's correction — an *unbiased* variance estimate. It matters here only as the constant $\frac{1}{n-1}$.)

SOLUTION

```
dbndiff2 = (1.0/(n-1)) * torch.ones_like(bndiff2) * dbnvar
```

15 dbndiff (part 2) (32, 64)

```
bndiff2 = bndiff**2
```

Power rule: local derivative 2 bndiff. This is the **second** path into `bndiff` (it also fed `bnraw` in Step 12), so accumulate:

$$\mathbf{dbndiff} \mathrel{+}= 2\,\mathrm{bndiff} \odot \mathbf{dbndiff2}.$$

SOLUTION

```
dbndiff += (2*bndiff) * dbndiff2
```

16 dhprebn (part 1) · dbnmeani

(32,64) · (64,)

```
bndiff = hprebn - bnmeani
```

Subtraction with `bnmeani` $(1, 64)$ broadcast over the batch.

- **dhprebn** (coefficient $+1$): $\mathbf{dhprebn} = \mathbf{dbndiff}$ — first of two paths (hprebn also makes the mean in Step 17).
- **dbnmeani** (coefficient -1, broadcast ⇒ sum over batch): $\mathbf{dbnmeani} = -\sum_{\text{rows}} \mathbf{dbndiff}$.

SOLUTION

```
dhprebn = dbndiff.clone()        # part 1 of 2
dbnmeani = (-dbndiff).sum(0)
```

17 dhprebn (part 2)

(32, 64)

```
bnmeani = 1/n * hprebn.sum(0, keepdim=True)
```

The batch mean. Local derivative of each summand is $\frac{1}{n}$, broadcast back over the rows. This is the **second** path into `hprebn`, so accumulate (Rule 6):

$$\mathbf{dhprebn}[i, j] \mathrel{+}= \frac{1}{n}\, \mathbf{dbnmeani}[j].$$

With both paths summed, `dhprebn` is finally complete.

SOLUTION

```
dhprebn += 1.0/n * (torch.ones_like(hprebn) * dbnmeani)
```

18 **dembcat · dW1 · db1** (32,30) · (30,64) · (64,)

```
hprebn = embcat @ W1 + b1
```

Linear layer 1 — Rule 4 again with $X = \text{embcat}$, $W = W_1$:

$$\mathbf{dembcat} = \mathbf{dhprebn}\, W_1^{\top}, \quad \mathbf{dW_1} = \text{embcat}^{\top}\, \mathbf{dhprebn}, \quad \mathbf{db_1} = \sum_{\text{rows}} \mathbf{dhprebn}.$$

SOLUTION

```
dembcat = dhprebn @ W1.T
dW1 = embcat.T @ dhprebn
db1 = dhprebn.sum(0)
```

19 **demb** (32, 3, 10)

```
embcat = emb.view(emb.shape[0], -1)
```

A pure `view` (reshape) — no arithmetic, no data moved, just a reinterpretation of $(32, 3, 10)$ as $(32, 30)$. The gradient is the same numbers, reshaped **back** to the original layout:

$$\mathbf{demb} = \text{reshape}\big(\mathbf{dembcat},\ (32, 3, 10)\big).$$

SOLUTION

```
demb = dembcat.view(emb.shape)
```

20 dC (27, 10)

```
emb = C[Xb]
```

The embedding lookup is a **gather**: row (k, j) of `emb` is the row `C[Xb[k,j]]` of the table. By Rule 5 the backward pass is a **scatter-add** — each `demb[k,j]` is added into the table row whose index was used. Because a character index appears many times across the batch, those gradients accumulate:

$$\mathbf{dC}[c] = \sum_{(k,j)\,:\,Xb[k,j]=c} \mathbf{demb}[k, j].$$

SOLUTION

```
dC = torch.zeros_like(C)
for k in range(Xb.shape[0]):
    for j in range(Xb.shape[1]):
        ix = Xb[k,j]
        dC[ix] += demb[k,j]
```

VECTORIZED ALTERNATIVE

The double loop is the clearest version. In production you'd write `dC.index_add_(0, Xb.view(-1), demb.view(-1, C.shape[1]))` — same scatter-add, no Python loop.

RESULT

Running `cmp(...)` on all 26 tensors prints **exact: True · maxdiff: 0.0** for every single one. You have re-implemented `loss.backward()` by hand.

2. Exercise 2 — cross-entropy in one go

Steps 1–8 backpropagated cross-entropy through eight tiny pieces. Done as a single mathematical expression instead, the gradient collapses into three lines and a famously clean formula.

Set-up

For one example with logit row $\ell \in \mathbb{R}^{27}$ and correct class y, the softmax probabilities and loss are

$$p_j = \frac{e^{\ell_j}}{\sum_k e^{\ell_k}}, \qquad L = -\log p_y.$$

(The max-subtraction for stability cancels in the ratio, so it never appears in the gradient.)

Derivation

Split the log of the quotient:

$$L = -\log p_y = -\ell_y + \log \sum_k e^{\ell_k}.$$

Differentiate with respect to a single logit ℓ_j. The first term contributes -1 only when $j = y$. The second term is the log-sum-exp, whose derivative *is* the softmax:

$$\frac{\partial}{\partial \ell_j} \log \sum_k e^{\ell_k} = \frac{e^{\ell_j}}{\sum_k e^{\ell_k}} = p_j.$$

Putting the two together gives the celebrated result

$$\boxed{\frac{\partial L}{\partial \ell_j} = p_j - \mathbf{1}[j = y]}$$

"the softmax probabilities, minus one at the true class." Averaging over the n examples in the batch adds the $\frac{1}{n}$:

$$\frac{\partial L}{\partial \ell_{ij}} = \frac{1}{n}\left(p_{ij} - \mathbf{1}[j = y_i]\right).$$

From formula to code

Read the boxed formula literally: start with the full softmax matrix, subtract 1 at each row's true class, divide by n.

SOLUTION — "MY SOLUTION IS 3 LINES"

```
dlogits = F.softmax(logits, 1)
dlogits[range(n), Yb] -= 1
dlogits /= n
```

EXACT: FALSE, APPROXIMATE: TRUE (MAXDIFF ≈ 6E-9)

Unlike Exercise 1, this won't be bit-exact — the one-shot formula and PyTorch's eight-step path accumulate floating-point rounding differently. A maxdiff around 10^{-9} is the correct, expected answer.

INTUITION

Each row of `dlogits` sums to (essentially) zero: probability is conserved, so pushing the true class up by some amount pushes the others down by the same total. The gradient says, plainly, *"raise the score of the right character, lower the rest, in proportion to how much mass they wrongly hold."*

3. Exercise 3 — batch-norm in one go

Steps 10–17 wound through bndiff, bndiff2, bnvar, bnvar_inv, bnraw and two means. Collapsing batch-norm into a single backward expression is the hardest derivation in the lecture — and the most rewarding. Here it is, in full.

Set-up (per neuron, over the batch)

Fix one of the 64 columns and let $x_1, \dots, x_n$ be its pre-activations across the batch (a column of `hprebn`). Forward:

$$\mu = \frac{1}{n}\sum_i x_i, \qquad \sigma^2 = \frac{1}{n-1}\sum_i (x_i - \mu)^2,$$

$$s = (\sigma^2 + \varepsilon)^{-1/2}, \qquad \hat{x}_i = (x_i - \mu)\, s, \qquad y_i = \gamma\, \hat{x}_i + \beta.$$

Here $s =$ `bnvar_inv`, $\hat{x} =$ `bnraw`, $\gamma =$ `bngain`. We are given $g_i = \partial L / \partial y_i$ (this is `dhpreact`) and want $\partial L / \partial x_i$ (this is `dhprebn`).

Derivation

Since $y_i = \gamma \hat{x}_i + \beta$, the gradient into the normalized values is $\partial L / \partial \hat{x}_i = \gamma g_i$. Every x_i influences *every* $\hat{x}_j$, through three channels — directly, through the shared mean μ, and through the shared variance σ^2:

$$\frac{\partial L}{\partial x_i} = \sum_j \frac{\partial L}{\partial \hat{x}_j}\frac{\partial \hat{x}_j}{\partial x_i}, \qquad \frac{\partial \hat{x}_j}{\partial x_i} = \left(\delta_{ij} - \frac{\partial \mu}{\partial x_i}\right) s + (x_j - \mu)\frac{\partial s}{\partial x_i}.$$

The two shared pieces are

$$\frac{\partial \mu}{\partial x_i} = \frac{1}{n}, \qquad \frac{\partial \sigma^2}{\partial x_i} = \frac{2}{n-1}(x_i - \mu) \;\; (\text{using } \textstyle\sum_k (x_k - \mu) = 0),$$

$$\frac{\partial s}{\partial x_i} = -\tfrac{1}{2}(\sigma^2 + \varepsilon)^{-3/2}\frac{\partial \sigma^2}{\partial x_i} = -\frac{s^3}{n-1}(x_i - \mu).$$

Substitute and sum over j:

$$\frac{\partial L}{\partial x_i} = s\frac{\partial L}{\partial \hat{x}_i} - \frac{s}{n}\sum_j \frac{\partial L}{\partial \hat{x}_j} - \frac{s^3}{n-1}(x_i - \mu)\sum_j \frac{\partial L}{\partial \hat{x}_j}(x_j - \mu).$$

Now use $\hat{x} = (x - \mu)s$ to replace $(x - \mu) = \hat{x}/s$ in the last term; two powers of s cancel, leaving everything in terms of $\hat{x}$ and s:

$$\frac{\partial L}{\partial x_i} = s\left[\frac{\partial L}{\partial \hat{x}_i} - \frac{1}{n}\sum_j \frac{\partial L}{\partial \hat{x}_j} - \frac{1}{n-1}\,\hat{x}_i \sum_j \frac{\partial L}{\partial \hat{x}_j}\,\hat{x}_j\right].$$

Finally put back $\partial L/\partial \hat{x} = \gamma g$ and factor out $\frac{1}{n}$:

$$\boxed{\frac{\partial L}{\partial x_i} = \frac{\gamma\, s}{n}\left[n\, g_i \;-\; \sum_j g_j \;-\; \frac{n}{n-1}\,\hat{x}_i \sum_j g_j\,\hat{x}_j\right]}$$

From formula to code

The box maps term-for-term onto Karpathy's "one (long) line," with $\gamma =$ `bngain`, $s =$ `bnvar_inv`, $g =$ `dhpreact`, $\hat{x} =$ `bnraw`, and the column-sums $\sum_j(\cdot)$ written as `.sum(0)`:

SOLUTION — "MY SOLUTION IS 1 (LONG) LINE"

```
dhprebn = bngain*bnvar_inv/n * (n*dhpreact - dhpreact.sum(0)
          - n/(n-1)*bnraw*(dhpreact*bnraw).sum(0))
```

EXACT: FALSE, APPROXIMATE: TRUE (MAXDIFF ≈ 9E-10)

As in Exercise 2, the collapsed formula differs from PyTorch's step-by-step path only by floating-point rounding — a maxdiff near 10^{-9} is exactly right.

READING THE THREE TERMS

$n\, g_i$ is the raw incoming gradient; subtracting $\sum_j g_j$ **re-centres** it (removes the mean component the normalization made unavailable); the last term **removes the variance component**, scaled by how far $\hat{x}_i$ sits from centre. Batch-norm's backward pass is literally "the gradient, with its mean and variance directions projected out."

4. Exercise 4 — putting it all together

Now we train the full network (200k steps, 200 hidden units) using *only* the hand-derived gradients — no `loss.backward()`. The forward pass uses the compact batch-norm, and the whole loop runs inside `torch.no_grad()` because we no longer need PyTorch's autograd graph.

The complete manual backward pass

This is Exercises 2 and 3 (the fast cross-entropy and the one-line batch-norm) stitched between the two linear layers and the embedding scatter-add from Exercise 1:

SOLUTION — FULL BACKWARD BLOCK

```
# cross-entropy (Exercise 2)
dlogits = F.softmax(logits, 1)
dlogits[range(n), Yb] -= 1
dlogits /= n
# Linear layer 2
dh  = dlogits @ W2.T
dW2 = h.T @ dlogits
db2 = dlogits.sum(0)
# tanh
dhpreact = (1.0 - h**2) * dh
# batch-norm (Exercise 3): gamma, beta, and the one-liner for the input
dbngain = (bnraw * dhpreact).sum(0, keepdim=True)
dbnbias = dhpreact.sum(0, keepdim=True)
dhprebn = bngain*bnvar_inv/n * (n*dhpreact - dhpreact.sum(0)
          - n/(n-1)*bnraw*(dhpreact*bnraw).sum(0))
# Linear layer 1
dembcat = dhprebn @ W1.T
dW1 = embcat.T @ dhprebn
db1 = dhprebn.sum(0)
# embedding scatter-add (Exercise 1, step 20)
demb = dembcat.view(emb.shape)
dC = torch.zeros_like(C)
for k in range(Xb.shape[0]):
    for j in range(Xb.shape[1]):
        ix = Xb[k,j]
```

```
        dC[ix] += demb[k,j]

grads = [dC, dW1, db1, dW2, db2, dbngain, dbnbias]
```

The update step

With gradients in hand, take the SGD step manually — note `grad` from our list, not `p.grad` from autograd:

SOLUTION — PARAMETER UPDATE

```
lr = 0.1 if i < 100000 else 0.01      # step LR decay
for p, grad in zip(parameters, grads):
    p.data += -lr * grad               # swole-doge: our own gradient
```

What changes versus the debug forward pass

- **Compact batch-norm.** Training uses `bnmean = hprebn.mean(0)` and `bnvar = hprebn.var(0, unbiased=True)` directly — same math as the chunked version, fewer intermediates.
- **No `retain_grad()`, no `.backward()`.** They exist in the earlier cells only so `cmp` can check you. Once verified, delete them; the loop runs under `with torch.no_grad():` for speed.
- **Calibrate batch-norm at the end.** After training, run the training set through once to record `bnmean` / `bnvar`, used as fixed statistics at inference (single-example prediction has no batch to normalize against).

THE PAYOFF

Trained entirely on hand-written gradients, the net reaches **train ≈ 2.07, val ≈ 2.11** — identical to the autograd version — and samples plausible names (*carmahzamille, khalaysie, jareen, quinn…*). You have built, and backpropagated, a neural net from scratch. 🥷
